Web-приложения и их архитектура: DOM

DOM-дерево. Теория

Немного про среды

JavaScript изначально был разработан для работы с браузерами. Его главная цель заключалась в добавлении интерактивности веб-страницам, то есть возможностей для взаимодействия с пользователями.

Сегодня JavaScript значительно развился иа стал полноценным языком программирования для решения действительно широкого спектра задач. Несмотря на то, что до сих пор он сохраняет свою роль флагмана фронтенд-разработки, его можно свободно запускать как на сервере, так и на любом другом устройстве, способном запускать «движок» JavaScript.

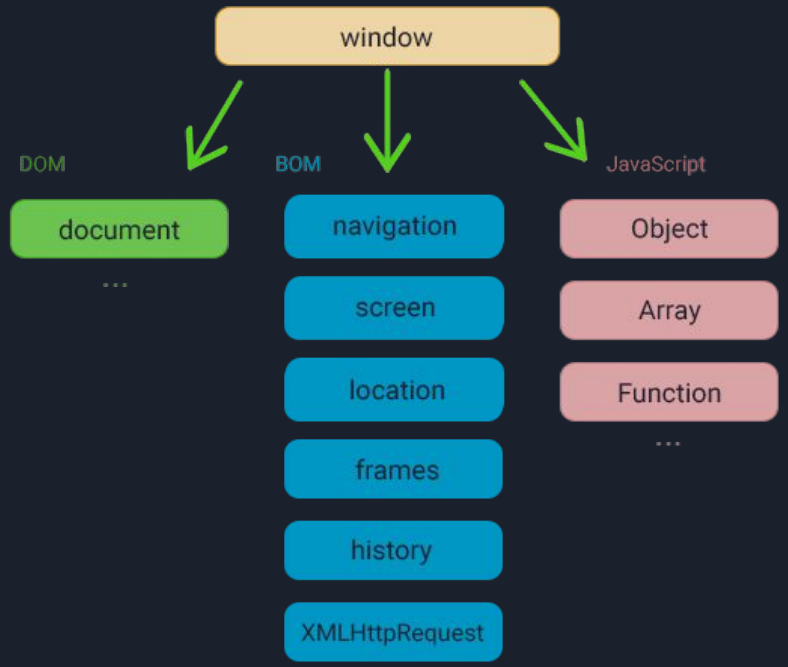
Каждая (хост) среда предоставляет свой собственный набор дополнительных возможностей (функций и объектов), уникальных для неё. Так, например, в случае браузера это объекты navigator, screen, document или функции setTimeout, setInterval, prompt, confirm и т. д. Когда JavaScript запускается в браузере, он автоматически получает доступ ко всем этим возможностям. Аналогичная ситуация и в других средах.

Среда браузера (Browser environment)

Вы можете получить доступ к API браузера, используя глобальный объект window. Более того, этот объект включает в себя как объектную модель документа (DOM), так и объектную модель браузера (BOM), а также объекты и функции JavaScript. В результате получается, что window является одновременно объектом среды браузера и глобальным объектом для JavaScript.

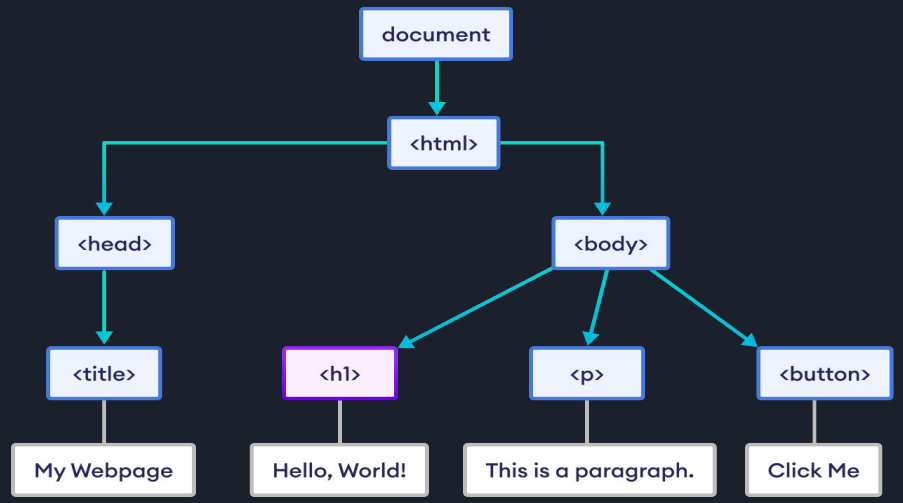
DOM — это объектная модель документа, создаваемая при загрузке веб-страницы на основе специальных объектов.

BOM — это набор инструментов, предоставляемых браузером.



Что такое DOM?

DOM (Document Object Model) — это древовидная структура, создаваемая браузером из HTML-кода, которая представляет страницу как набор узлов и объектов. JavaScript использует DOM API для динамического изменения структуры, стилей и контента веб-страниц, добавляя интерактивность. Основные операции включают поиск элементов, изменение их содержимого, стилей и обработку событий.

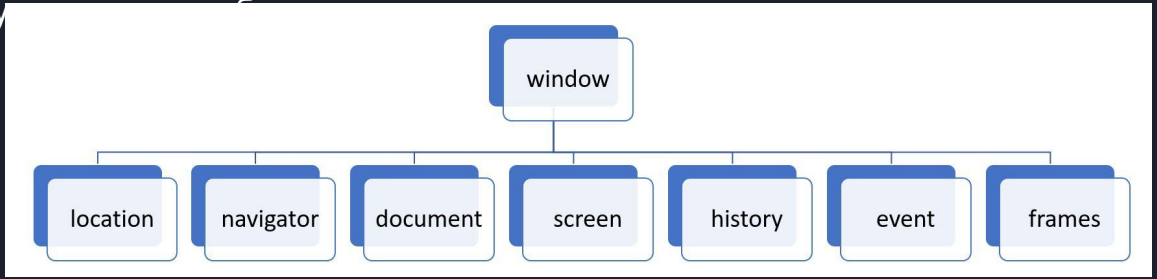


Browser Object Model (BOM)

BOM (метка порядка байтов) — это набор функций и объектов, предоставляемых браузером.

Например, из упомянутого ранее объекта navigator можно получить много информации о приложении, запустившем скрипт (например, браузер, операционную систему и многое другое). Объект location содержит информацию о текущем URL-адресе и позволяет выполнять с ним некоторые манипуляции.

BOM является частью общей спецификации HTML. Следовательно, спецификация HTML — это не только описание того, как работает сам язык разметки HTML, но и ряд инстру



DOM-дерево

Основой любого HTML-документа являются теги.

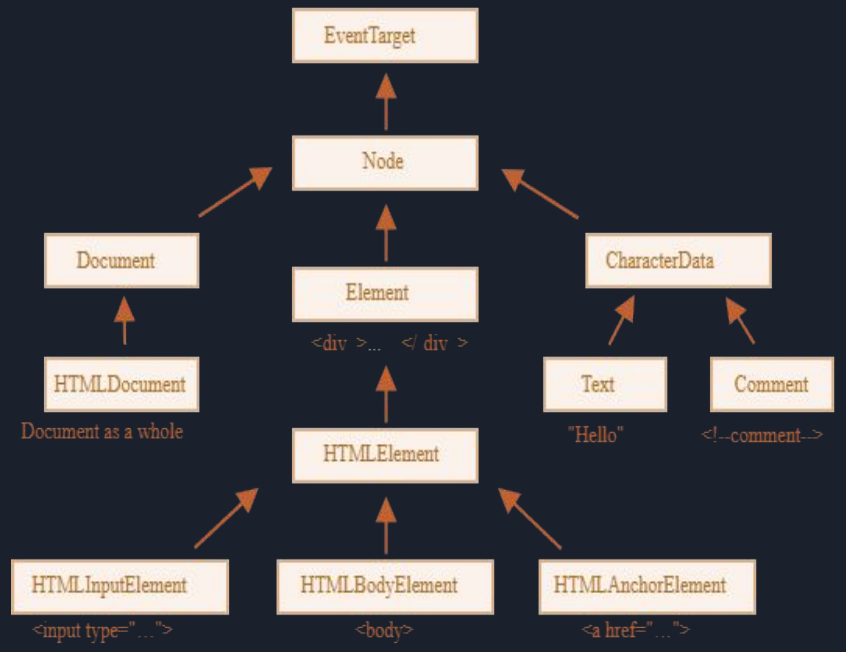
Более того, согласно объектной модели документа, каждый тег является объектом. Также каждый тег является элементом в DOM-дереве (подробнее об этом позже). Вложенные теги являются «дочерними» элементами родительских элементов. Последние, в свою очередь, могут иметь свои собственные вложенные элементы и так далее. Таким образом, строится иерархия (дерево) элементов HTML-страницы.

Фрагменты текста и комментарии также являются узлами DOM-дерева. При этом содержимое тега <script> также является текстом с точки зрения документа.

Ядра(Nodes)

В DOM-дереве существует два основных типа узлов: обычные узлы и узлы элементов (или просто элементы). Абсолютно всё, что вы видите на странице HTML-документа, состоит из узлов (частные случаи которых представлены элементами).

В WebAPI узлы представлены интерфейсом Node. Все остальные узлы и элементы наследуются от этого интерфейса (опять же, включая узлы комментариев, текстовые узлы, фрагменты документа (DocumentFragment) и т. д.). Каждый объект узла имеет свойства для получения коллекции (итерируемого псевдомассива) дочерних узлов (childNodes), типа узла (nodeType) и многих других. Кроме того, объекты узлов содержат ряд методов, включая методы для вставки (appendChild), удаления (removeChild), клонирования (cloneNode) узлов и т. д.



Элементы (Elements)

Элементы — это все наши теги. В целом, в разработке приложений в 99,99% случаев мы хотим работать только с элементами, а не со всеми узлами.

Как и узлы, элементы имеют собственное представление в WebAPI, а именно интерфейс Element. Этот интерфейс предоставляет расширенный набор свойств, включая свойства, предназначенные для работы исключительно с элементами. Например, свойство children возвращает коллекцию (итерируемый псевдомассив), содержащую все дочерние элементы. Свойство nextElementSibling содержит ссылку на следующий элемент в дереве или null, если его нет. Использование его «старшего брата» nextSibling в большинстве случаев неудобно, поскольку в качестве значения можно получить абсолютно любой узел (даже текстовый узел или комментарий).

Элементы составляют основу интерфейса и возможностей работы с ним. Мы можем прикреплять к ним обработчики событий, изменять их содержимое и внешний вид. Для назначения обработчиков существует встроенный метод addEventListener. Для установки встроенных стилей существует специальное свойство style.

Основные элементы DOM

С помощью DOM мы можем делать с внешним видом и содержимым наших элементов все, что захотим. Однако, прежде чем выполнять какие-либо операции с ними, их необходимо получить.

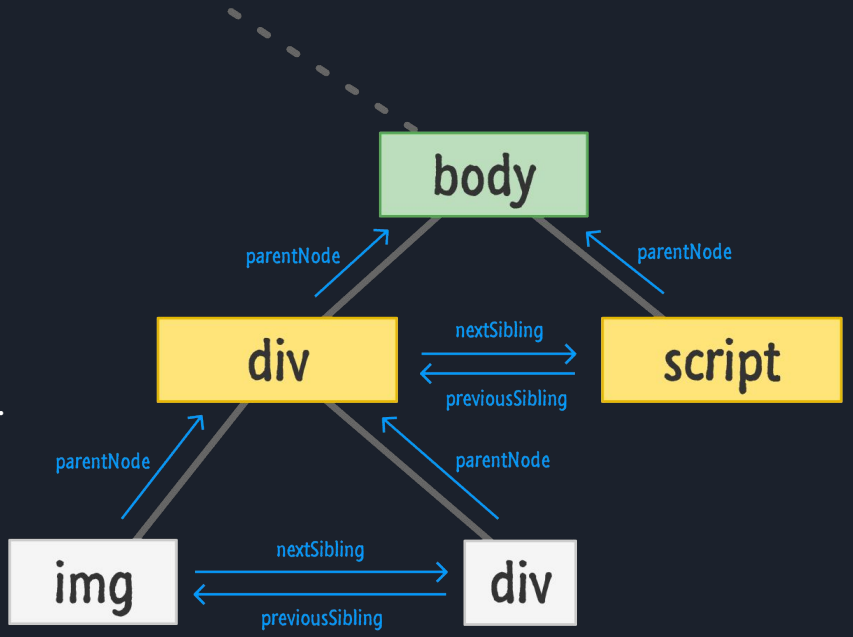
Объект `document` является корневым и входным объектом дерева DOM. Доступ к другим узлам дерева невозможен без доступа к document. Кроме того, document имеет несколько свойств, содержащих ссылки на ключевые элементы дерева DOM. Это свойства documentElement, head и body. Они содержат ссылки на элемент html (`<html></html>`), элемент head (`<head></head>`) и элемент body (`<body></body>`) нашей страницы соответственно. Это сделано, очевидно, для упрощения доступа к элементам, необходимым в каждом HTML-документе. Доступ ко всем остальным элементам осуществляется несколько иначе. Однако одно всегда остается неизменным — любой элемент на странице можно получить через `document`.

Взаимодействие с DOM. Получение элементов

Навигация по DOM

Существует множество встроенных инструментов для навигации по DOM:

- Свойства узла для получения родительского узла/элемента: `parentNode` и `parentElement` соответственно.
- Свойства элемента для получения соседних узлов/элементов: `previousSibling`, `nextElementSibling` и т. д.
- Свойства элемента для получения дочерних узлов/элементов: `childNodes`, `children`, `firstChild`, `lastElementChild` и т. д.



Поиск/получение элементов DOM-дерева. Document.getElementById & Element.getElementsByClassName & Element.getElementsByTagName

```html
<div id="field">
    <div class="cell"></div>
    <div class="cell"></div>
    <div class="cell"></div>
    <div class="cell"></div>
    <div class="cell"></div>
</div>

<footer id="footer">
</footer>
```

get-element-by-id.js

get-element-by-id.js > ...

```js
const field = document.getElementById('field');
const unique = document.getElementById('unique');
const footer = document.getElementById('footer');
```

```html
<div id="field">
    <header class="field__item"></header>

    <div class="field__item"></div>
    <div class="field__item"></div>
    <div class="field__item"></div>

    <footer class="field__item"></footer>
</div>
```

get-elements-by-*.js

get-elements-by-*.js > ...

```js
const field = document.getElementById('field');

const fieldItems = field.getElementsByClassName('field__item');
const fieldCells = field.getElementsByClassName('cell');
const fieldDivs = field.getElementsByTagName('div');
```

Поиск/получение элементов DOM-дерева. Document.getElementById & Element.getElementsByClassName & Element.getElementsByTagName

`.getElementById()` возвращает ссылку на элемент по его идентификатору (ID); идентификатор является строкой, которая может быть использована для идентификации элемента; она может быть определена при помощи атрибута id в HTML или из скрипта.

`.getElementsByClassName()` возвращает массивоподобный (итерируемый) объект всех дочерних элементов, соответствующих всем из указанных имён классов. В случае вызова по отношению к объекту 'document', поиск происходит по всему документу, включая корневой элемент.

`.getElementsByTagName()` метод возвращает живую коллекцию элементов HTMLCollection , учитывая имя тэга. Поиск осуществляется в поддереве указанного элемента, в результат поиска не попадает сам элемент, в поддереве которого осуществлялся поиск. Возвращает живой список, который автоматически обновляется при изменении DOM.

Поиск/получение элементов DOM-дерева. Element.querySelector & Element.querySelectorAll

Результатом вызова метода `querySelector` будет первый элемент среди дочерних элементов в любой вложенности, соответствующий переданному CSS-селектору, или null, если такого элемента нет.

Результатом вызова метода `querySelectorAll` будет коллекция (итерируемый псевдомассив) дочерних элементов любой вложенности, соответствующих переданному CSS-селектору. Если таких элементов нет, мы получим пустую коллекцию.

```html
<div id="field">
    <header class="field__item"></header>

    <div class="field__item"></div>
    <div class="field__item"></div>
    <div class="field__item"></div>

    <footer class="field__item"></footer>
</div>
```

query-selectors.js

query-selectors.js > ...

```js
const field = document.querySelector('#field');
const fieldItemDivs = field.querySelectorAll('div.field__item');
const fieldFooter = field.querySelector('footer.field__item');
```

“Живые” и “неживые” коллекции

```
const booksList = document.querySelector('.books');
const liveBooks = booksList.children;

// Удалим первую книгу
liveBooks[0].remove();
// Получим 2
console.log(liveBooks.length);
// Получим элемент book--two, который теперь стал первым в коллекции
console.log(liveBooks[0]);
```

```
const notLiveBooks = document.querySelectorAll('.book');

// Удалим первую книгу
notLiveBooks[0].remove();
// Получим 3
console.log(notLiveBooks.length);
// Получим ссылку на удалённый элемент book--one
console.log(notLiveBooks[0]);
```

Events

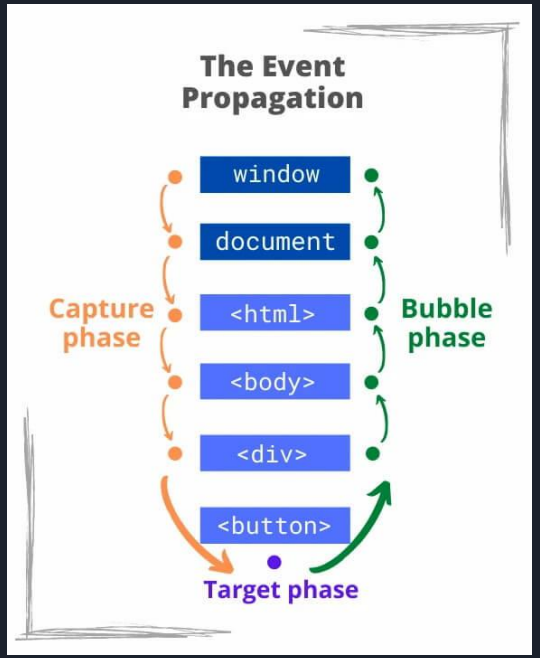
Events (browser events)

Event— это сигнал от браузера о том, что что-то произошло. Все элементы DOM (а также document и window) испускают эти сигналы. Некоторые из наиболее распространенных событий:

- click/dblclick (mouse) — щелчок/двойной щелчок левой кнопкой мыши по элементу (на сенсорных устройствах touch/double-tap);
- contextmenu (mouse) — щелчок правой кнопкой мыши по элементу;
- mouseover/mouseout (mouse) — наведение/перемещение курсора мыши над элементом;
- keydown/keyup (keyboard) — нажатие/отпускание клавиши на клавиатуре;
- focus/blur (control) — фокус/потеря фокуса на элементе;
- input/change (control) — изменение значения поля ввода/завершение изменения значения;
- submit (control) — отправка формы <form></form>;
- DOMContentLoaded (document) - HTML-код загружен и обработан, DOM полностью построен.

Событийные фазы (Event phases)

- Capturing Phase (Фаза погружения/захвата): Событие движется от window вниз по DOM-дереву к целевому элементу (цели). Обработчики, зарегистрированные с параметром { capture: true }, срабатывают здесь.
- Target Phase (Фаза цели): Событие достигло элемента, на котором оно было инициировано (например, нажатая кнопка). Обработчики, привязанные к этому элементу, выполняются.
- Bubbling Phase (Фаза всплытия): Событие движется от цели вверх к window, последовательно вызывая обработчики у родителей. Это поведение по умолчанию для большинства событий.



Фаза таргета

На этапе определения целевого элемента мы получаем доступ к нему через свойство event.target.

Стоит отметить, что в некоторых случаях event.target может быть равен event.currentTarget, но в конечном итоге эти свойства сообщают нам разные вещи, поэтому их всегда следует использовать по назначению. Event.currentTarget всегда ссылается на элемент, на котором был срабатывал обработчик. Event.target — это элемент, который инициирует событие.

Event.stopPropagation()

Как можно увидеть со схемы, событие проходит три стадии. По сути, стадию таргета можно считать частью “всплытия”, так как, по сути, при каком-то событии у нас происходит “распространение” события наверх - в родительские элементы. Чтобы этого не происходило, существует такой метод, как stopPropagation() - он останавливает “распространение” события, т.е прерывает стандартный процесс обработки события.

Event handlers

У каждого элемента есть возможность прослушивать различные события - клик, нажатие клавиши, двойной клик. Для обработки подобных событий и существуют event handler.

Event handler, по сути - функция, которая вызывается при срабатывании какого-то события.

```
<button class="primary-button" onclick="alert('Clicked!')">Click me!</button>
```

```
<button class="primary-button" onclick="console.log('first'); console.log('second');">Click me!</button>
```

```
<button class="primary-button" onclick="doSmth()">Click me!</button>
```

Использование свойств элемента

Аналогичным образом, вы можете прикреплять обработчики событий, используя встроенные свойства элементов DOM. По умолчанию каждый элемент содержит все возможные свойства для обработки событий со значениями null. Это означает, что события на этом элементе не обрабатываются (конечно, если только обработчики не прикреплены с помощью addEventListener, но об этом позже). Для обработки события вам просто нужно присвоить функцию соответствующему свойству. Например:

```
1   <button class="primary-button">Click me!</button>
```

script.js

script.js > ...

```
1   const button = document.querySelector('button.primary-button');
2   button.ondblclick = function() {
3       console.log(`You've just clicked on the button twice!`);
4   }
```

target.addEventListener()

Иногда бывает ситуация, когда на созданный элемент DOM нужно поставить обработку какого-то события. В этом случае на помощь приходит такой метод, как target.addEventListener.

Метод addEventListener() интерфейса EventTarget устанавливает функцию, которая будет вызываться каждый раз, когда указанное событие будет достигать цели.

```
1   button.addEventListener('click', () => {
2       console.log(`You've just clicked on the button!`);
3   }, { once: true, passive: false, capture: false });
```

Удаление обработчиков

Существует два способа удалить обработчик, назначенный с помощью addEventListener:

- Используйте метод `removeEventListener`, который удаляет обработчик по ссылке.
- При назначении обработчика передайте объект со значением, равным `true`, в качестве третьего параметра. После первого срабатывания обработчик будет автоматически удален.

Метод `removeEventListener`(`eventName`, `eventHandler`, `?capture`) принимает три параметра, два из которых точно такие же, как у метода `addEventListener`. Третий параметр требует указания фазы, из которой удаляется обработчик. То есть, если обработчик был назначен на этап погружения, то он должен быть удален из той же фазы. В противном случае ничего не будет удалено.

Важно отметить, что `removeEventListener` ожидает ссылку на функцию обработчика, которая была назначена ранее. То есть нам нужна та же ссылка, которая использовалась в addEventListener. Вы не сможете использовать новую функцию с идентичным телом.

Handler Object

В дополнение к описанным ранее способам использования метода `addEventListener`, есть ещё один. В качестве второго параметра можно передать не функцию, а объект. Если такое событие произойдет, внутренний вызов будет выглядеть так: `object.handleEvent(event)`. То есть в объекте необходимо реализовать метод `handleEvent`, который будет вызываться при срабатывании события. Например:

```
1  button.addEventListener('dblckick', {
2      handleEvent(event) {
3          console.log(`You've just clicked on the button twice!`);
4      }
5  });
```

Чтобы удалить такой обработчик в `removeEventListener`, второй параметр снова должен быть ссылкой на тот же объект, который использовался в `addEventListener`. При удалении такого обработчика необходимо также учитывать фазу.

Event object

Каждый раз, когда браузер отправляет сигнал о произошедшем событии, он инициализирует и отправляет вместе с ним специальный объект события. Этот объект содержит информацию о том, что произошло. Например, какая клавиша была нажата на клавиатуре или в каких координатах находился курсор мыши в момент срабатывания события клика. Объект события всегда передается в функцию обработчика в качестве первого параметра.

Объекты событий могут быть разными. Например, упомянутый ранее объект события нажатия клавиши клавиатуры отличается от объекта события нажатия клавиши мыши. Однако каждый из них содержит ряд общих свойств и методов. Некоторые из этих свойств:

- `event.target` – элемент, который является инициатором события (или целевой элемент);
- `event.currentTarget` – элемент, для которого непосредственно указан обработчик события;
- `event.type` – тип события в виде строки (например, '`click`').

Также через `this` можно получить ссылку на элемент, для которого назначен обработчик. Значение this – это ссылка на этот элемент. Однако это не всегда срабатывает, например,

Event object

Помимо встроенных свойств, объект события также имеет несколько методов. Рассмотрим наиболее популярные из них:

- `event.preventDefault()` — отменяет действие по умолчанию (например, добавление символа в поле ввода при нажатии клавиши или появление контекстного меню в браузере при нажатии «`contextmenu`»);
- `event.stopPropagation()` — останавливает дальнейшее распространение события. То есть, после вызова этого метода событие завершит «погружение» (или «всплытие»), и ни один из оставшихся обработчиков других событий не будет вызван;
- `event.stopImmediatePropagation()` — то же самое, что и `event.stopPropagation()`, но дополнительно предотвращает вызов других обработчиков для текущего элемента. То есть, если на одном элементе одновременно зависли три обработчика «`click`», вызов `event.stopImmediatePropagation()` в одном из них предотвратит вызов остальных.

Делегирование событий

Иногда мы сталкиваемся с ситуациями, когда в определенной области находится множество элементов одного типа, каждый из которых должен обрабатывать одно и то же событие с помощью одного алгоритма. Мой любимый пример такой ситуации — игра «Сапер». Для каждой клетки нам нужно назначить два обработчика: один для щелчка левой кнопкой мыши и один для щелчка правой кнопкой. Конечно, мы можем прикрепить к каждой клетке два отдельных обработчика, но только представьте, сколько таких обработчиков будет на поле 10x10. А на поле 20x20? Немного многовато.

Такие ситуации не так уж редки, и, естественно, для них есть отличное решение — делегирование событий. Его суть сводится к тому, что для обработки одного события на всех элементах одновременно нам нужно назначить контейнеру только один обработчик. В обработчике, основываясь на информации о целевом элементе (свойство event.target), мы выполняем или не выполняем дальнейшие операции. Часто в таких ситуациях в качестве ориентира используется класс элемента. Например: event.target.classList.contains('cell').

Изменения в DOM

Создание ядер (nodes)

Допустим, мы хотим создать не просто статичную страницу, но добавить в неё немного динамики. Допустим, мы хотим добавить какой-то элемент. Как это сделать?

Объект `document` имеет встроенный метод `createElement(tagName, ?options)`, который, очевидно, необходим для создания элементов. Этот метод принимает два аргумента, но в большинстве случаев нативный JavaScript использует только первый. В результате мы получаем специальный объект нашего элемента.

```
1   const divElement = document.createElement('div');
2   const unorderedListElement = document.createElement('ul');
```

Добавление ядер

Обычно мы создаём новые узлы и элементы, чтобы позже вставить их в документ. Они не вставляются в DOM-дерево автоматически при создании. Однако существует множество различных способов сделать это, и наиболее современные из них следующие:

- `node.append(...nodes or strings)` — вставляет переданные узлы/строки после всех дочерних узлов узла;
- `node.prepend(...nodes or strings)` — вставляет переданные узлы/строки перед всеми дочерними узлами узла;
- `node.before(...nodes or strings)` — вставляет переданные узлы/строки непосредственно перед узлом;
- `node.after(...nodes or strings)` — вставляет переданные узлы/строки сразу после узла;
- `node.replaceWith(...nodes or strings)` — вставляет переданные узлы/строки вместо узла.

Добавление ядер

```html
<div id="card">
    <h1>Header</h1>
    <p>Some text…</p>
    <!-- some important comment -->
</div>
```

nodes-insertion.js

nodes-insertion.js > ...

```js
const card = document.getElementById('card');
const anotherImportantComment = document.createComment('another important comment');
const hintElement = document.createElement('h4');
card.append(anotherImportantComment, 'and some text');
card.prepend(hintElement);
card.before('before card...');
card.after('after card...');
```

Добавление ядер

```
1   before card...
2   <div id="card">
3       <h4></h4>
4       <h1>Header</h1>
5       <p>Some text…</p>
6       <!-- some important comment -->
7       <!--another important comment-->
8       and some text
9   </div>
10  after card...
11
```

nodes-insertion.js ×

nodes-insertion.js > ...

```
1   const card = document.getElementById('card');
2   const anotherImportantComment = document.createComment('another important comment');
3   const hintElement = document.createElement('h4');
4   card.append(anotherImportantComment, 'and some text');
5   card.prepend(hintElement);
6   card.before('before card...');
7   card.after('after card...');
8
```

Вставка HTML-фрагментов

Мы рассмотрели, как вставлять узлы и обычные строки в документ, но что, если мы захотим передать, например, следующую строку в качестве значения для вставки:

«<h1>Неожиданно!</h1>»

Нарушит ли это структуру документа? Приведёт ли это к появлению нового HTML-элемента? Нет. И нет. Описанные ранее методы безопасно вставляют строки в DOM. Это необходимо для защиты документа от вмешательства в его структуру. Вы также можете безопасно вставлять или изменять текст в документе, используя свойство `node.textContent`. Однако стоит помнить, что новое значение этого свойства заменит всё ранее существовавшее содержимое узла.

Когда вам нужно вставить HTML-фрагмент, самый простой способ — использовать свойство `element.innerHTML`. При изменении этого свойства строка, переданная в качестве нового значения, преобразуется в HTML перед заменой предыдущего содержимого элемента. Однако вам нужно использовать это свойство осторожно и всегда проверять, что именно вводится, чтобы предотвратить XSS-атаки. Вы можете проверить это вручную (что не всегда является лучшим вариантом, поскольку вы можете что-то упустить) или воспользоваться готовым решением.

Ещё пара методов по вставке

В дополнение к свойству `element.innerHTML`, элементы имеют метод `element.insertAdjacentHTML(where, html)`, который помогает добиться практически того же самого. Метод принимает два обязательных аргумента: специальную строку, указывающую точное место вставки нового HTML-фрагмента относительно элемента, и сам HTML-код. Возможные значения для аргумента where:

- “`beforebegin`” (node.before);
- “`afterbegin`” (node.prepend);
- “`beforeend`”(node.append);
- “`afterend`” (node.after).

Кроме того, есть еще два очень похожих метода:

- `element.insertAdjacentText(where, text)`
- `element.insertAdjacentElement(where, element)`

Однако они используются крайне редко.

Удаление ядра

Удаление узлов очень просто. Лучший способ сделать это — использовать метод `node.remove()`. Помимо него, существует также метод `node.removeChild(childNode)`, но он не так удобен в использовании. В случае `node.remove()` мы вызываем метод непосредственно на удаляемом узле. В случае `node.removeChild(childNode)` нам обязательно нужно получить родительский узел и вызвать метод на нём, передав удаляемый узел в качестве параметра. Второй вариант сейчас используется почти никогда.

Помимо явного удаления узлов, можно прибегнуть к другим методам. Как упоминалось ранее, при изменении значений свойств `node.textContent` и `element.innerHTML` старое содержимое узла перезаписывается новым. Это также удаляет узлы. Однако эти варианты, скорее всего, будут полезны только в ситуациях, когда нам нужно полностью очистить содержимое нашего узла (то есть удалить сразу «пакет» дочерних узлов).

Клонирование (копирование)

Создать клон узла очень просто. Достаточно использовать встроенный метод `node.cloneNode(?deep)`. Этот метод принимает один необязательный параметр, указывающий, следует ли выполнять глубокое клонирование объекта. Если параметр не указан или равен `false`, то клон будет представлять только сам узел без его предков.

При перемещении узлов они не клонируются, а перемещаются сами. То есть, при попытке добавить их в разные части документа, элементы каждый раз «перетаскиваются» с одного места на другое. Чтобы создать несколько одинаковых элементов, их необходимо клонировать или создавать заново каждый раз.

Ещё про вставку элементов

До появления современных методов вставки, удаления и замены узлов, таких как append, before, replaceWith и т. д., использовался ряд других методов. Сегодня они практически не применяются. Основные причины этого: меньшая гибкость и более трудоемкое использование. Однако мы не можем не упомянуть их, поскольку они все еще встречаются в старом коде.

`node.appendChild(newNode)` – вставляет newNode после всех дочерних узлов узла;

`node.removeChild(nodeToRemove)` – удаляет nodeToRemove из дочерних узлов узла;

`node.insertBefore(newNode, nextSibling)` – вставляет newNode перед nextSibling внутри узла;

`node.replaceChild(newNode, oldChildNode)` – заменяет oldChildNode на newNode среди дочерних узлов.

Вы заметите, что раньше для вставки, замены и даже удаления элементов всегда приходилось выполнять операцию через родительский элемент.

Отображение элементов

Довольно часто в процессе разработки приложений нам необходимо отображать один и тот же элемент по-разному в зависимости от состояния. Или просто динамически изменять внешний вид как всего элемента, так и его отдельных частей. Все это, конечно же, можно реализовать с помощью JavaScript и встроенных свойств и методов элементов.

Существует три основных способа изменения внешнего вида элемента с помощью JavaScript:

- Изменение атрибутов элемента;
- Изменение набора классов элемента;
- Изменение стилей элемента.

В ситуациях, когда речь идет об изменении внешнего вида элемента с помощью JavaScript или CSS, всегда отдавайте предпочтение CSS! Используйте JavaScript только тогда, когда CSS вам недостаточно.

Работа с атрибутами элемента

Для работы с атрибутами интерфейс Element предоставляет ряд встроенных методов:

- `element.hasAttribute(attrName)` — проверяет, есть ли у элемента атрибут с тем же именем;
- `element.getAttribute(attrName)` — получает значение атрибута элемента;
- `element.setAttribute(attrName, value)` — устанавливает значение атрибута;
- `element.removeAttribute(attrName)` — удаляет атрибут элемента.

Кроме того, существует свойство `element.attributes`, которое возвращает весь список атрибутов элемента в виде коллекции (итерируемый псевдомассив). Для каждого атрибута из этой коллекции можно получить имя (поле name) и значение (поле value).

В большинстве случаев вам не нужно работать с атрибутами напрямую. Наиболее распространенные случаи работы с ними: изменение состояния поля ввода на disabled или readonly, отображение состояния checked в флажках и т. д.

Работа с классами элемента

Для работы с классами интерфейс Element предоставляет два встроенных свойства: `className` (строка, содержащая все классы элемента, разделенные пробелом) и `classList` (коллекция (итерируемый псевдомассив) всех имен классов элемента в виде строк). В подавляющем большинстве случаев второй вариант будет предпочтительным способом работы с классами. В дополнение к более удобному формату представления набора классов (в виде коллекции, а не строки со значениями, разделенными пробелами), свойство `classList` содержит ряд вспомогательных методов:

- `element.classList.add('class')` - добавить класс с именем class к элементу;
- `element.classList.remove('class')` - удалить класс элемента с именем class;
- `element.classList.toggle('class')` - если элемент уже имеет класс с именем class, то удалить его, если нет, то добавить;
- `element.classList.contains('class')` - определить, имеет ли элемент класс с именем class.

Работа со стилями элементами

К любому HTML-элементу можно добавить атрибут `style`. Получить значение этого атрибута можно либо обычным способом (используя метод `getAttribute`), либо напрямую обратившись к свойству элемента с тем же именем. В JavaScript нам, естественно, удобнее работать со свойствами. Более того, это свойство содержит не просто строковое представление наших стилей (как атрибут), а целый объект. Изменяя этот объект, мы сразу же меняем внешний вид элемента. Например, установка `element.style.fontSize = '24px'` установит параметр размера шрифта равным 24px. Важно отметить, что стандартные имена свойств CSS с дефисами (стиль `kebab-case`) преобразуются в имена в стиле `camelCase`. При этом имена с префиксами браузеров пишутся в стиле `PascalCase` (например, `MozBorderRadius`, `WebkitAnimation` и т. д.).

Чтобы удалить один из стилей, ранее записанных в объект стиля, вам нужно не просто удалить это свойство (например, удалить element.style.color), а присвоить ему пустую строку или значение null. В противном случае свойство просто не будет удалено.

Добавление нескольких стилей за раз

С помощью специального свойства `element.style.cssText` можно задать сразу несколько стилей для элемента. В качестве входных данных это свойство должно принимать строку, содержащую допустимый CSS-код в формате: «свойство 1: значение; свойство 2: смысл...». Таким образом, в этой строке можно ввести все необходимые стили в привычном нам формате, даже одним подходом. Однако здесь стоит понимать, что при изменении значения этого свойства ВСЕ стили, ранее указанные через объект `style`, перезаписываются (даже если новые стили не совпадают со старыми). Поэтому лучше использовать свойство `element.style.cssText` только тогда, когда мы точно знаем, что ничего необходимого не будет потеряно (например, при создании элемента).

Кроме того, несколько стилей можно задать одновременно, используя уже знакомый метод element.setAttribute, где первым параметром будет строка «style», а вторым — строка формата: «свойство 1: значение; свойство 2: смысл...».

На этом всё. Основные статьи, рекомендуемые к ознакомлению на следующем слайде

https://habr.com/ru/articles/554776/

https://developer.mozilla.org/ru/docs/Web/API/EventTarget/addEventListener

https://developer.mozilla.org/ru/docs/Web/API/Element/getElementsByTagName

В принципе, рекомендую пройтись по всем методам JS в документации: https://developer.mozilla.org/ru